





OBJETOS DISTRIBUIDOS

CHADWICK CARRETO ARELLANO



6 DE OCTUBRE DE 2026

ROSALES BARRAZA ERICK EFREN
7CM1

Objetos distribuidos con Java RMI

Quinta parte: los riesgos que aparecen en el uso normal de una tienda con objetos remotos

Antecedentes

En la cuarta parte el almacén se repartió entre tres servidores que se hablaban por sockets TCP. El siguiente cambio de enfoque fue pasar a objetos distribuidos: en lugar de mandar mensajes, el cliente invoca métodos de un objeto que vive en otra máquina, como si fuera local. Primero se armó en JavaScript un mecanismo que imitaba esa idea, con proxy, registro y despachador, pero eso no era RMI. Se aclaró que hacía falta RMI de verdad.

RMI (Remote Method Invocation) es la tecnología de Java para invocar métodos de objetos que están en otra JVM. El cliente obtiene un stub desde un registro, llama sus métodos y la plataforma se encarga de serializar los argumentos, enviarlos por red y devolver el resultado. Cada método remoto está obligado a declarar RemoteException, justamente porque la red puede fallar.

Antes de llegar a RMI hubo dos versiones intermedias en JavaScript. La primera era un guion que mostraba siete riesgos uno por uno, cada uno con su etiqueta. No cumplía lo pedido, porque los riesgos se exhibían en lugar de aparecer solos, y se eliminó. La segunda fue una tienda normal, con cinco clientes y un almacén remoto, sobre un mecanismo de objetos remotos hecho a mano y una red simulada en memoria. Esa tienda es la versión anterior con la que se compara este trabajo.

Planteamiento del problema

El pedido tenía dos condiciones. La primera, usar java.rmi de verdad, con interfaces Remote, objetos exportados con UnicastRemoteObject, un Registry y el recolector de basura distribuido. La segunda, que los riesgos de los objetos distribuidos no se mostraran en un guion que los exhibiera uno por uno, sino que salieran como saldrían en cualquier ejecución de una aplicación común.

Eso trajo un reto práctico: RMI corre sobre TCP y, en una sola computadora, la red local casi nunca falla. Había que provocar fallas realistas, como mensajes perdidos, respuestas lentas y reinicios del servidor, sin tocar el código de la aplicación para que esta se comportara como cualquier otra.

Solución

Se reescribió la tienda con RMI. Cinco clientes, cada uno en su propio hilo, hacen doce compras cada uno contra un almacén remoto. El servidor publica en el registro dos objetos, Almacen y Fabrica, y cada compra pide a la fábrica un Carrito remoto nuevo. El código de la aplicación está escrito de la forma más directa posible, como lo haría alguien que apenas aprende RMI:

- Si una llamada lanza RemoteException, se reintenta hasta tres veces.
- Primero se consulta el stock y después se descuenta, en dos llamadas separadas.
- Cada cliente busca el stub una sola vez, al empezar, y lo usa hasta el final.
- Los carritos remotos se crean en cada compra y nadie los libera.

- Un método de reporte devuelve el keySet() de un mapa.

La inestabilidad de la red se inyecta con una fábrica de sockets propia, que RMI permite registrar con RMISocketFactory. Cada escritura se demora unos milisegundos, de vez en cuando mucho más, y con una probabilidad baja se pierde el mensaje cerrando la conexión. Las solicitudes se pierden con 3% de probabilidad y las respuestas con 5%. El cliente espera como máximo 40 ms por una respuesta. Además, en algunas corridas el servidor se reinicia por mantenimiento. Al final de cada jornada se hace un cierre de caja que compara lo que los clientes creen haber vendido con lo que el almacén realmente descontó.

Implementación

- Almacen, Carrito y Fabrica son las interfaces remotas, y AlmacenImpl, CarritoImpl y FabricaImpl sus implementaciones. Venta es un registro serializable que viaja del cliente al servidor.
- CarritoImpl implementa Unreferenced para contar cuántos carritos libera el recolector de basura distribuido.
- RedInestable es la RMISocketFactory que demora y pierde mensajes. El registro queda fuera de las fallas para que el problema esté en los objetos de la tienda y no en el arranque.
- Tienda arranca el registro y el servidor, lanza a los clientes, programa el reinicio y imprime el cierre de caja.

Los dos métodos del almacén que más importan para los resultados son descontar, que no está protegido ante llamadas concurrentes, y horasConVentas, que devuelve una vista del mapa:

AlmacenImpl.java: descontar y horasConVentas

```
@Override
public int descontar(String producto, int cantidad) {
    int nuevo = stock.get(producto) - cantidad;
    stock.put(producto, nuevo);
    bitacora.merge(producto, cantidad, Integer::sum);
    return nuevo;
}

@Override
public Collection<Integer> horasConVentas() {
    Map<Integer, Integer> porHora = new HashMap<>();
    for (Venta venta : ventas) {
        int hora = venta.fecha().toInstant().atZone(ZoneId.systemDefault()).getHour();
        porHora.merge(hora, venta.cantidad(), Integer::sum);
    }
    return porHora.keySet();
}
```

El cliente repite cualquier llamada que falle y arma cada compra con varias llamadas remotas seguidas:

Tienda.java: reintento y secuencia de una compra

```
static <T> T reintentar(Llamada<T> llamada) throws RemoteException {
    RemoteException ultimo = null;
    for (int intento = 0; intento < 3; intento++) {
```

```
        try {
            return llamada.ejecutar();
        } catch (RemoteException e) {
            ultimo = e;
        }
    }
    throw ultimo;
}

Carrito carrito = reintentar(() -> fabrica.crearCarrito());
reintentar(() -> carrito.agregar(producto));
int disponible = reintentar(() -> almacen.stock(producto));
if (disponible < cantidad) { ... continue; }
reintentar(() -> almacen.descontar(producto, cantidad));
reintentar(() -> almacen.registrarVenta(new Venta(nombre, producto, cantidad, new
Date())));
```

La pérdida de mensajes se logra en la escritura del socket. Si el mensaje que se pierde es una solicitud, el servidor nunca se entera. Si es una respuesta, el servidor ya ejecutó el método:

RedInestable.java: escritura que demora y pierde mensajes

```
@Override
public void write(byte[] b, int off, int len) throws IOException {
    demorar();
    if (perder(esSolicitud)) {
        socket.close();
        throw new IOException("mensaje perdido en la red");
    }
    out.write(b, off, len);
}
```

El reinicio retira los objetos del servidor y los vuelve a publicar con un estado nuevo:

Tienda.java: reinicio del servidor

```
UnicastRemoteObject.unexportObject(almacenActual, true);
UnicastRemoteObject.unexportObject(fabricaActual, true);
Thread.sleep(40);
publicar();

static void publicar() throws RemoteException {
    almacenActual = new AlmacenImpl(STOCK_INICIAL, bitacora);
    fabricaActual = new FabricaImpl();
    registro.rebind("almacen", UnicastRemoteObject.exportObject(almacenActual, 0));
    registro.rebind("fabrica", UnicastRemoteObject.exportObject(fabricaActual, 0));
}
```

Cambios respecto a la versión anterior

Pasar de la tienda en JavaScript a RMI no fue una traducción línea por línea. Cambió la plataforma, el mecanismo de llamadas remotas y también el modo en que fallan las cosas. La tabla resume cada cambio:

Aspecto	Versión anterior (JavaScript)	Versión con Java RMI
Plataforma	Node.js; se ejecuta directo con node objetos/tienda.js.	Java 21; hay que compilar con javac (npm run rmi:build) y ejecutar con java -cp rmi/out tienda.Tienda.
Objetos remotos	Mecanismo casero: Servidor.registrar y despachar, y un proxy dinámico (crearProxy).	RMI real: interfaces Remote, UnicastRemoteObject.exportObject, Registry y stubs que genera la plataforma.
Contrato	El proxy aceptaba cualquier nombre de método y lo validaba hasta ejecutar.	Interfaces Almacen, Carrito y Fabrica. Cada método declara RemoteException y el compilador obliga a manejarla.
Red	RedSimulada en memoria, con promesas y un temporizador por llamada.	Sockets TCP reales en la máquina local. RedInestable, una RMISocketFactory, demora y pierde mensajes en cada escritura.
Latencia y espera	De 5 a 30 ms por sentido; espera máxima de 50 ms por llamada.	De 2 a 11 ms por escritura, con 6% de picos de 45 a 89 ms; espera de lectura de 40 ms.
Pérdida de mensajes	4% de las solicitudes y 6% de las respuestas, por llamada.	3% y 5% por cada escritura. Una llamada RMI hace varias escrituras, así que la pérdida real por llamada es mayor.
Concurrencia	Un solo hilo con asincronía. Las carreras ocurrían entre llamadas separadas por un await.	Hilos reales: cinco hilos cliente y los hilos con que RMI atiende cada llamada. HashMap y ArrayList sin sincronizar.
Referencias	El carrito se identificaba por un nombre (carrito-N) y el cliente armaba un proxy con él.	Fabrica.crearCarrito devuelve el stub de un objeto exportado. El objeto viaja por referencia.
Liberación	Opciones ttlMs y barrer(), que la tienda no usaba. Se contaban los objetos vivos.	Recolector distribuido de RMI e interfaz Unreferenced. Se cuentan carritos creados y liberados.
Reinicio	reiniciar() vaciaba el registro y se volvían a registrar los mismos nombres. El mismo proxy volvía a funcionar, con estado nuevo.	unexportObject y una nueva publicación. Los stubs viejos fallan para siempre con NoSuchObjectException, porque el cliente busca el stub una sola vez.
Reintentos	El proxy reintentaba hasta 2 veces, solo ante un timeout.	La función reintentar de la aplicación: 3 intentos ante cualquier RemoteException.
Errores	ErrorTimeout y ErrorRemoto, definidos por el proyecto.	Jerarquía de RemoteException de Java: ConnectIOException, UnmarshalException y NoSuchObjectException.
Serialización	JSON. La fecha llegaba como texto, Map y Set quedaban vacíos y una referencia circular lanzaba error.	Serialización de Java. Venta es Serializable y su Date viaja bien. El fallo se mueve a NotSerializableException al devolver un keySet().
Reporte final	resumenPorHora fallaba con "getHours is not a function".	horasConVentas devuelve una Collection y falla con NotSerializableException.
Cierre de caja	Terminaba con "Objetos remotos vivos en el servidor".	Termina con "Carritos remotos creados y liberados por el recolector distribuido".
Azar	Generador propio (mulberry32) compartido por todo el programa.	java.util.Random con semilla; cada cliente usa la semilla más su número. Se fija con la variable SEMILLA.
Arranque y salida	El proceso de Node termina solo.	Se fija java.rmi.server.hostname en 127.0.0.1 y se llama a System.exit(0), porque los hilos de RMI mantienen vivo el proceso.

Lo que se mantuvo igual para que la comparación fuera justa: el inventario inicial (60 manzanas, 50 peras y 70 bananas), los cinco clientes con doce compras cada uno, la probabilidad de reinicio del servidor (35%), el estilo ingenuo del cliente y el cierre de caja. En cuanto a archivos:

- Archivos nuevos en la carpeta rmi: Almacen, AlmacenImpl, Carrito, CarritoImpl, Fabrica, FabricaImpl, Venta, RedInestable y Tienda, todos en el paquete tienda, más fuentes.txt con la lista para compilar.
- Archivo modificado: package.json, con dos scripts nuevos, rmi:build y rmi.
- Archivos sin cambios: los de la carpeta objetos (almacen.js, orb.js, red.js y tienda.js), que se conservan como la versión en JavaScript.
- Las opciones de idempotencia, lista blanca y expiración que tenía orb.js no tienen equivalente en la versión de Java. No se portaron porque la tienda tampoco las usaba.

Resultados

Se hicieron nueve corridas, cada una con una semilla distinta para la red, y las anomalías aparecieron sin que el programa se lo propusiera.

Excepciones observadas

Los clientes vieron siete combinaciones de excepción y causa. Lo que más importa de cada una es si el método llegó a ejecutarse en el servidor:

Excepción observada	Qué pasó	¿Se ejecutó el método?
ConnectIOException <- IOException ("mensaje perdido en la red")	Se perdió la solicitud, o un mensaje del establecimiento de la conexión.	No. Reintentar es seguro.
ConnectIOException <- EOFException	La conexión se cerró antes de terminar el saludo inicial.	No. La llamada no llegó a enviarse.
UnmarshalException <- EOFException	Se perdió la respuesta y el servidor cerró la conexión.	Sí. El cliente no lo sabe y reintenta.
UnmarshalException <- SocketTimeoutException ("Read timed out")	La respuesta tardó más de 40 ms y el cliente dejó de esperar.	Sí, casi siempre. El mismo problema.
UnmarshalException <- IOException ("mensaje perdido en la red")	Se perdió un mensaje mientras el cliente procesaba la respuesta.	Normalmente sí.
NoSuchObjectException	El servidor se reinició y el stub apunta a un objeto que ya no existe.	No. Y seguirá fallando.
UnmarshalException <- NotSerializableException (HashMap$KeySet)	El servidor no puede serializar el valor que el método devuelve.	Sí, pero el resultado nunca llega. Falla siempre.

Anomalías en los datos y en el comportamiento

- En ocho de las nueve corridas hubo al menos un producto en el que los clientes creían haber vendido menos de lo que el almacén descontó. La excepción fue una corrida en la que el servidor se reinició casi al principio y casi no hubo ventas. En una corrida, por ejemplo, creían haber vendido 55 manzanas y el

almacén descontó 63. La diferencia viene de las respuestas perdidas: el servidor descuenta, el cliente no se entera y vuelve a pedir.

- Hubo compras que el cliente dio por fallidas y que sí se descontaron. En una corrida con reinicio, los clientes creían haber vendido 0 manzanas y el almacén había descontado 2: una explicación probable es que el descuento se ejecutó y el registro de la venta, justo después, ya encontró el objeto sin existir. La compra son dos llamadas separadas, y si la segunda falla queda a medias.
- En una de las nueve corridas el stock de manzanas terminó en -3. Dos clientes consultaron el stock casi al mismo tiempo, ambos vieron que alcanzaba y los dos descontaron.
- El servidor se reinició en cuatro de las nueve corridas. A partir de ese momento, todos los clientes recibieron NoSuchObjectException en cada compra hasta el final, porque conservaban el stub anterior. En la versión de JavaScript esto no pasaba, porque el nombre se resolvía de nuevo en cada llamada. En RMI el stub lleva el identificador del objeto, y ese identificador deja de existir. En esas corridas el stock volvió a su valor inicial sin avisar a nadie.
- El método horasConVentas falló en las tres corridas en que se revisó su salida completa, con UnmarshalException causada por NotSerializableException: java.util.HashMap$KeySet. La vista que devuelve keySet() no es serializable, aunque el mapa sí lo sea.
- Se crearon, por ejemplo, 60 carritos y el recolector distribuido había liberado 43 al terminar. En otras corridas fueron 66 y 45. Los demás siguen exportados hasta que el recolector local y el arrendamiento los alcancen, y ese momento no es predecible.

Comparación con la versión en JavaScript

Los problemas de fondo se repitieron, pero algunos cambiaron de forma. Los descuentos duplicados, los reinicios y los objetos sin liberar aparecieron en las dos versiones. El stock negativo apareció en JavaScript en dos de las corridas revisadas (con -2 y -10) y en Java en una de nueve (con -3). El error de la fecha convertida a texto desapareció, porque la serialización de Java conserva el tipo Date, pero apareció uno equivalente, el de la vista que no se puede serializar. La diferencia más importante es la del reinicio. En JavaScript la aplicación se recuperaba, aunque con el estado perdido. En RMI, un cliente que conserva el stub viejo queda inutilizado hasta que vuelve a consultar el registro.

Conclusiones

RMI hace que una llamada remota se escriba igual que una local, y esa es la parte engañosa. Obliga a declarar RemoteException, pero atrapar la excepción no resuelve el problema de fondo: cuando una llamada falla, el cliente no sabe si el método se ejecutó. Reintentar a ciegas, que es lo primero que se le ocurre a cualquiera, fue la causa de los descuentos duplicados.

Ninguno de los problemas necesitó condiciones extremas. Bastaron un tiempo de espera de 40 ms y una pérdida de mensajes de entre 3% y 5% para que cada corrida terminara con cuentas que no cuadraban. También se vio que algunos riesgos dependen de decisiones de diseño ordinarias, como devolver un keySet() o consultar el stock y descontarlo en dos llamadas.

Para reducir estos riesgos, el siguiente paso sería hacer las operaciones idempotentes con un identificador por llamada, juntar consultar y descontar en una sola operación atómica, volver a pedir el stub cuando aparezca NoSuchObjectException, devolver copias serializables como new HashSet<>(mapa.keySet()), y liberar de forma

explícita los objetos remotos que ya no se usan. Ninguna de esas medidas está en esta versión, porque el objetivo era observar los problemas.

Una limitación de este trabajo es que el servidor y los clientes corren en la misma JVM. Las llamadas sí viajan por sockets TCP reales y pasan por la serialización de RMI, pero las fallas de red son simuladas, no vienen de una red física.